

Mobile and Embedded Computing

Lecture 3: agent-assisted coding, Flutter widgets, async & isolates

Who's teaching this course

Dinu-Ștefan Rusu

dinu_stefan.rusu@upb.ro

Microsoft Teams course channel

rusudinu.com

45+

Flutter apps shipped

400k

installs across them

Appeared in, spoken at & partnered with



TODAY

What you should leave with



Work with an agent

Prompt it with real context, then review its output like any other pull request



Read a widget tree

Composition over inheritance, and what actually rebuilds when state changes



Keep the frame budget

One frame = $1 \div \text{refresh rate}$: 8.3 ms on a 120 Hz phone, not a fixed 16 ms



Async vs isolates

When to await, when to spawn, and how Dart's model differs from Go's

AGENT-ASSISTED CODING

Coding with an agent in the loop

The 2026 tool landscape, and the review discipline it requires

The 2026 tool landscape



Agentic CLIs

Claude Code, Codex CLI, Gemini CLI: they run in your repository, edit files and run your tests



IDE assistants

Cursor, GitHub Copilot, Windsurf, JetBrains AI: completion plus in-editor chat



Cloud agents

Long-running tasks against a checkout; what comes back is a pull request you review



Review bots

CodeRabbit and friends comment on your PR before a human ever opens it



Tool access (MCP)

Agents reach your files, shell, database and APIs through a tool protocol



One layer down

All of them are shells over a handful of frontier models; the shell is what differs

The category moves every few months. What does not move is the part below: how you brief it, and how you check it.

What agents are good at, and where they fail

Where it helps

- + Boilerplate: scaffolding, data classes, test fixtures, glue code
- + Explaining unfamiliar code, stack traces and build errors
- + Mechanical refactors that touch twenty files at once
- + The test you were quietly going to skip
- + A first draft of a task you are unsure how to start

Where it costs you

- Plausible but wrong APIs: Flutter's surface moves fast
- Security holes: hardcoded keys, unvalidated input, open rules
- Code that ignores conventions you never told it about
- Silent drift away from the package versions you are actually on
- The error paths and edge cases nobody asked for

An agent produces confident output whether or not it is correct. Everything it writes still lands in the commit log under your name.

Prompting: give it context, not wishes

Name the files, packages and versions the change has to fit into

Break multi-part work into numbered steps: one concern per prompt

Ask for the shape of the output: a diff, a test, a table, a plan first

State your conventions explicitly; it cannot guess your codebase

Then read the diff yourself.

```
// Weak  
add pagination to the list
```

```
// Better  
In lib/ui/feed_page.dart, convert the  
ListView to ListView.builder and paginate:  
20 items per page, fetch the next page when  
the user is 200 px from the bottom, show a  
spinner row while loading.
```

```
Conventions: Dio for HTTP, no setState  
outside a State class, const where possible.  
Add a widget test for the loading row.
```

Review it like any other pull request



Read every line

If you cannot explain it in review, you cannot ship it, or defend it at the exam



Check the security surface

Hardcoded keys, unvalidated input, permissive database rules (→ Lecture 9)



Test the edges

Empty list, no network, expired token: the paths nobody thought to ask for



Check the API is current

Models learn from old docs: `firebase_ai`, not the deprecated `google_generative_ai`



Keep the slices small

One concern per branch and PR; generated code is only reviewable in slices



You own it

“The agent wrote it” is not a defense in code review, and not one here either

FLUTTER WIDGETS

Everything is a widget

The tree, composition, state, and what actually rebuilds

Everything is a widget, and three trees

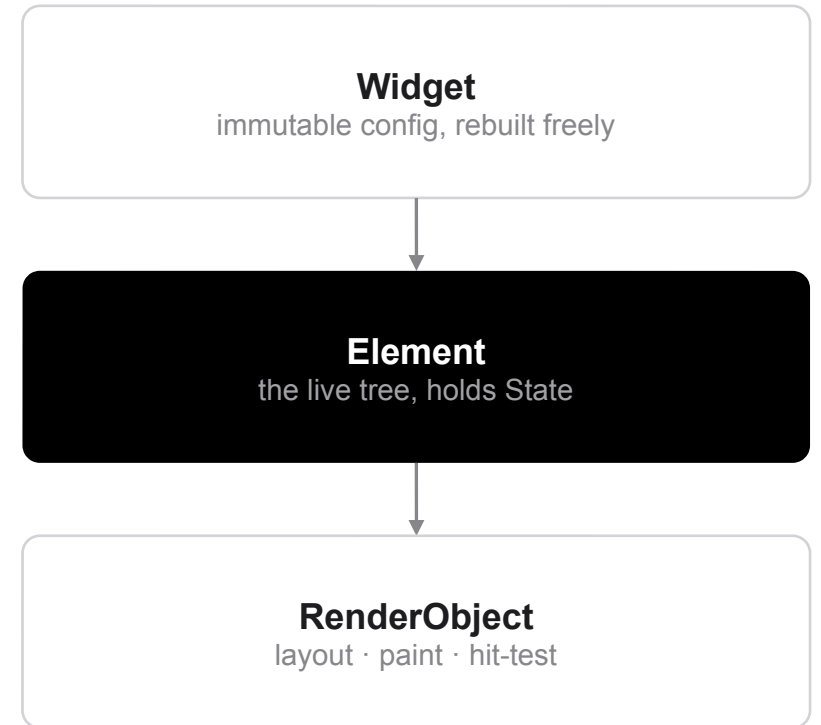
A widget is an immutable description of a piece of UI. Cheap to create, created and thrown away constantly.

You compose them: a Scaffold holds a Column, which holds a Text and an ElevatedButton. That nesting is the widget tree.

Behind it Flutter keeps two more. The element tree is the live instance graph: it holds State. The render tree does layout, paint and hit-testing.

You write the first tree. Flutter maintains the other two.

Rebuilding a widget is not redrawing the screen. It is handing Flutter a new description of what the screen should be.



Composition, not inheritance

```
// You do not subclass Text to give it padding
// and a border. You wrap it.

Padding(
  padding: const EdgeInsets.all(16),
  child: DecoratedBox(
    decoration: BoxDecoration(
      border: Border.all(color: Colors.black12),
    ),
    child: const Text('Hello'),
  ),
)
```

Each widget does one thing. Layout comes from nesting them, not from a widget with fifty properties.

That is why trees get deep; deep trees are normal and cheap.

Extract a subtree into its own named widget as soon as it repeats, or as soon as build() stops fitting on your screen.

Small widgets also rebuild in smaller pieces, which is the next slide.

Complex UIs are built out of simple widgets. There is no inheritance hierarchy to learn, only a catalog of widgets to compose.

StatelessWidget: configuration only

```
class Greeting extends StatelessWidget {  
  const Greeting({super.key, required this.name});  
  
  final String name; // every field is final  
  
  @override  
  Widget build(BuildContext context) {  
    return Text('Hello, $name');  
  }  
}
```

It does not "build once".

A StatelessWidget rebuilds whenever its parent rebuilds, or whenever something it depends on changes.

What it does not have is mutable state of its own: everything it needs arrives through its constructor.

Give it a const constructor whenever you can. That is what lets Flutter skip it later.

Stateless means "owns no state", not "is never rebuilt".

StatefulWidget and setState

```
class CounterView extends StatefulWidget {  
  const CounterView({super.key});  
  
  @override  
  State<CounterView> createState() => _CounterViewState();  
}  
  
class _CounterViewState extends State<CounterView> {  
  int _count = 0;          // the mutable state lives here  
  
  @override  
  Widget build(BuildContext context) => Column(children: [  
    Text('Count: $_count'),  
    ElevatedButton(onPressed: () => setState(() => _count++),  
      child: const Text('Add'))),  
  ]);  
}
```

Two classes, one widget

CounterView is still immutable configuration: it is recreated on every parent rebuild.

_CounterViewState is the object that survives those rebuilds. Your mutable fields go there.

setState mutates the field and marks the element dirty; Flutter rebuilds that subtree on the next frame.

Name the class for what it is.

What actually rebuilds

Flutter does not rebuild the whole widget tree each frame. `setState` marks one element dirty, and only that subtree rebuilds.

On rebuild, the new widgets are matched against the existing elements by `runtimeType` first, then by key.

Match → the element and its State are reused. No match → the old element is discarded and its State goes with it.

```
// Same instance every build → Flutter skips the subtree  
const Text('Total'),
```

Keys

`ValueKey('email')`: identity from a value

`ObjectKey(item)`: identity from an object

`UniqueKey()`: never matches, so it forces a fresh element

`GlobalKey`: reach a widget from anywhere, and use it sparingly

You need a key when list items are reordered, inserted or removed. Otherwise State follows position, not item.

Builds are designed to be cheap. Widgets are throwaway descriptions; elements and State are what persist.

The layout widgets you'll use every day

	What it does	The knobs you actually turn
Container	padding, margin, decoration and size in one box	padding · margin · decoration · alignment
Row / Column	lay children out along one axis	mainAxisAlignment · crossAxisAlignment · mainAxisAlignment
Expanded / Flexible	divide the leftover space in a Row or Column	flex: Expanded takes all, Flexible may take less
Stack / Positioned	overlay children; the first child paints at the bottom	top · left · right · bottom
SizedBox / ConstrainedBox	impose a size, or a min/max constraint	width · height · constraints
ListView / GridView	scroll a list or a grid, lazily	.builder · .separated · scrollDirection

Reach for Padding and SizedBox before Container. Container bundles several widgets; use it when you need most of them.

Making it scroll, lazily

```
// Eager: every child is built up front.
ListView(children: items.map(ItemTile.new).toList())

// Lazy: only the visible slice is ever built.
ListView.builder(
  itemCount: items.length,
  itemBuilder: (context, i) => ItemTile(
    key: ValueKey(items[i].id),
    item: items[i],
  ),
)
```

`ListView.separated` is the same thing with a `Divider` between items.

`GridView.builder` is the same idea in two dimensions.

`SingleChildScrollView` scrolls exactly one child and builds all of it: fine for a form, wrong for a feed.

`CustomScrollView` composes slivers when you need a collapsing app bar above a list.

Use a lazy list by default. If the list can grow with your data, it has to be a `.builder`.

CONCURRENCY

Async, isolates and the frame budget

One thread, one event loop, and when you need a second heap

One frame budget = $1 \div \text{refresh rate}$

16.7 ms

60 Hz

older devices, power-saving mode

11.1 ms

90 Hz

common mid-range in 2026

8.3 ms

120 Hz

most 2026 flagships, and they switch on the fly

Inside that budget the UI thread has to handle input, run `build()`, lay out, paint and hand the frame to the GPU.

Overrun it and the frame is dropped: that is the stutter users call jank. Repeated overruns show up as continuous stutter.

Do not hard-code 16 ms.

The event loop and the microtask queue

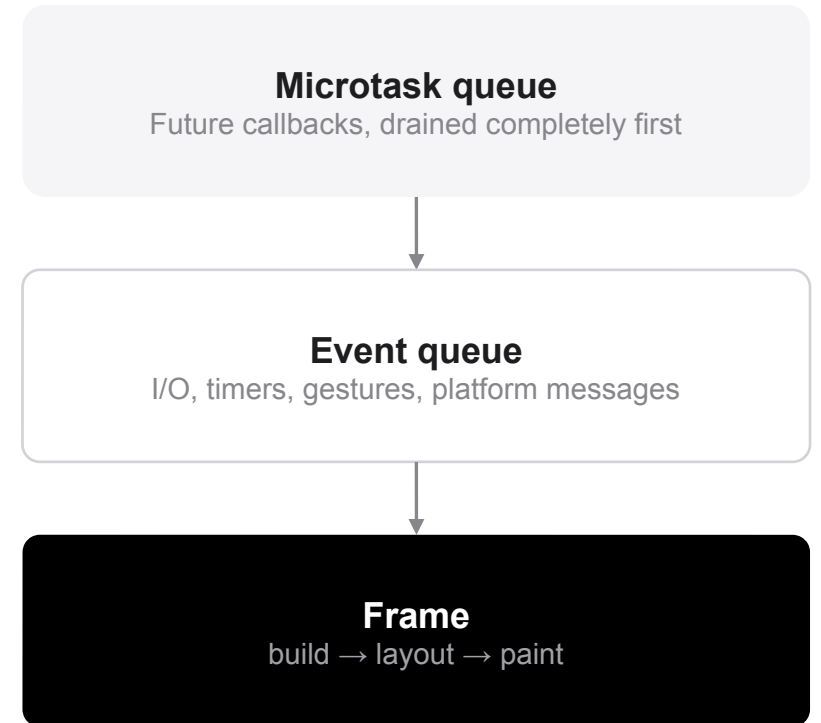
Your app runs on a single thread inside one isolate, the main isolate, and that isolate owns the UI.

That thread runs an event loop: take one event, run it to completion, drain the entire microtask queue, then take the next event.

`await` starts nothing. It registers the rest of your function as a continuation and hands control back to the loop.

Nothing else runs while your code runs. A 400 ms function is 400 ms of dropped frames.

Concurrent, not parallel. One thread, interleaving work that is mostly waiting.



Futures, async/await, and error handling

```
Future<User> loadUser(String id) async {  
  final res = await dio.get('/users/$id'); // suspends here  
  return User.fromJson(res.data);  
}  
  
Future<void> _refresh() async {  
  try {  
    _user = await loadUser(widget.id);  
  } on DioException catch (e) { // expected failure  
    _error = 'Network failed: ${e.message}';  
  } catch (e, stack) { // anything else  
    _error = 'Unexpected error';  
    FirebaseCrashlytics.instance.recordError(e, stack);  
  } finally {  
    if (mounted) setState(() => _loading = false);  
  }  
}
```

Errors inside async code

A failure inside async code does not throw at the call site; it completes the Future with an error.

await rethrows it, so try/catch/finally behaves exactly as it does in synchronous code. That is the whole point of await.

Without a try, the error escapes as an unhandled async error, which users rarely report.

Always check mounted before setState after an await: the widget may already have left the tree.

Streams: many values, over time

```
// A Future is one value, later.  
final user = await loadUser(id);  
  
// A Stream is many values, over time.  
Stream<int> ticker() async* {  
  for (var i = 0; ; i++) {  
    await Future.delayed(const Duration(seconds: 1));  
    yield i;  
  }  
}  
  
await for (final tick in ticker()) { print(tick); }
```

In a widget, use `StreamBuilder`: it subscribes, rebuilds on each event and cancels for you.

Streams are how WebSockets, sensor feeds and database snapshots reach your UI.

→ Lecture 8, WebSockets · → Lecture 11, sensors over MQTT

Cancel every subscription you create by hand in `dispose()`.

One value → Future. Many → Stream. Both ride the same event loop.

Async is not parallel

```
// This function is async. It still freezes the UI.  
Future<int> sumTo(int n) async {  
    var total = 0;  
    for (var i = 0; i < n; i++) {  
        total += i;          // pure CPU, it never yields  
    }  
    return total;           // the loop never got a turn  
}
```

Typical offenders: decoding a large JSON payload, image resize and filtering, encryption, compression, sorting hundreds of thousands of items.

async only helps when something else is doing the waiting: a socket, a disk, a timer.

Pure CPU work has nothing to wait for, so it never returns to the event loop.

Cooperative multitasking only works if every task yields.

Rule of thumb: waiting on I/O → async/await. Burning CPU for longer than a frame → isolate.

compute(): the one-call isolate

```
import 'package:flutter/foundation.dart';

// Must be a top-level or static function.
List<Photo> parsePhotos(String body) {
  final list = jsonDecode(body) as List<dynamic>;
  return list.map(Photo.fromJson).toList();
}

Future<List<Photo>> fetchPhotos() async {
  final res = await http.get(url);          // I/O → await
  return compute(parsePhotos, res.body);    // CPU → isolate
}
```

`compute()` spawns a short-lived isolate, runs one function, sends the result back and shuts it down.

Arguments and results must be sendable: primitives, Strings, Lists and Maps of them, `TransferableTypedData`.

It costs a few milliseconds to spawn, and the payload is copied both ways, so do not use it for 2 ms of work.

`compute()` covers most isolate work in application code. The next slide covers the cases it does not.

Isolate.spawn, SendPort, ReceivePort

```
Future<int> sumInIsolate(int n) async {  
    final inbox = ReceivePort();  
    await Isolate.spawn(_worker, [inbox.sendPort, n]);  
    final total = await inbox.first as int; // one message  
    inbox.close();  
    return total;  
}  
  
// Runs in a new isolate: own heap, own event loop.  
void _worker(List<Object> msg) {  
    final send = msg[0] as SendPort;  
    final n = msg[1] as int;  
    var total = 0;  
    for (var i = 0; i < n; i++) { total += i; }  
    Isolate.exit(send, total); // send and terminate  
}
```

The two ports

ReceivePort is your inbox. SendPort is a handle other isolates can post to.

Messages are deep-copied between heaps: nothing is shared, so nothing needs a lock.

Isolate.exit hands the result over without copying it, then ends the isolate.

Use this when the worker is long-lived or streams results back; use compute() otherwise.

Async vs isolates, precisely

	async / await	Isolate
Threads	one, interleaved on the UI thread	one per isolate, scheduled by the OS
Memory	one heap, shared by everything	a separate heap per isolate
Talking	just call the function	send messages over ports; values are copied
Good for	I/O: network, disk, database, timers	CPU: parse, decode, encrypt, compress, sort
Data races	impossible: one thing runs at a time	impossible: no shared mutable memory
Ordering races	possible	possible: replies can arrive out of order

No shared memory means no data races, but it does not mean no races. Two replies can still arrive in an order you did not plan for. And the number of isolates is not capped by core count: the OS schedules them like any other work.

Dart isolates vs Go goroutines

Dart: no shared memory, message passing

```
final inbox = ReceivePort();
await Isolate.spawn(producer, inbox.sendPort);

await for (final item in inbox) { // consumer
  if (item == null) break;
  print(item);
}

void producer(SendPort out) {
  for (var i = 0; i < 5; i++) out.send(i);
  out.send(null);
}
```

Go: shared memory, synchronized by channels

```
ch := make(chan int)

go func() { // producer
  for i := 0; i < 5; i++ {
    ch <- i
  }
  close(ch)
}()

for item := range ch { // consumer
  fmt.Println(item)
}
```

Same task, opposite guarantee. Dart copies messages between separate heaps, so sharing is impossible. Go shares one heap and uses channels to pass ownership; keeping that discipline is up to the programmer. → Lecture 12, Go & gRPC

Before next week



Recap

Agents draft; you brief, review, test and own the result

Widgets are immutable descriptions; only dirty subtrees rebuild, matched by runtimeType then key

One frame budget = $1 \div \text{refresh rate}$, not a fixed 16 ms

await is not a thread; an isolate is one, minus shared memory



This week

Lab: the counter, then a ListView.builder feed with keys

Wrap every await in your app in try / catch / finally

Move one CPU-heavy call into compute() and measure the frame times



Read more

docs.flutter.dev/ui/widgets-intro

dart.dev/language/async ·
dart.dev/language/concurrency

api.flutter.dev: Isolate, compute, StreamBuilder

go.dev/tour/concurrency